SAÉ 3.01 (R3.09)
Fonctions de hachage
Stocker un mot de passe

bruno.colombel@univ-amu.fr

BUT Informatique
IUT d'Aix-Marseille, site d'Arles

2026—2027

Fonctions de hachage

Sécurisation des mots de passe

Sommaire

Fonctions de hachage

Sécurisation des mots de passe

Fonction de hachage

Une fonction de hachage H prend en entrée un bit-string de n'importe quelle taille et retourne un haché (`digest`) ou empreinte de taille fixe n

$$h : \{0;1\}^* \longrightarrow \{0;1\}^n$$

Fonction de hachage – propriétés

Pre-image resistance (One-Way) OWHF

Soit y, il est difficile de retrouver x tel que

$$h(x) = y$$

Fonction de hachage – propriétés

2nd Pre-image resistance (weak-collision resistant CRHF)

Soit x, il est difficile de trouver $x' \neq x$ tel que :

$$h(x') = h(x)$$

Fonction de hachage – propriétés

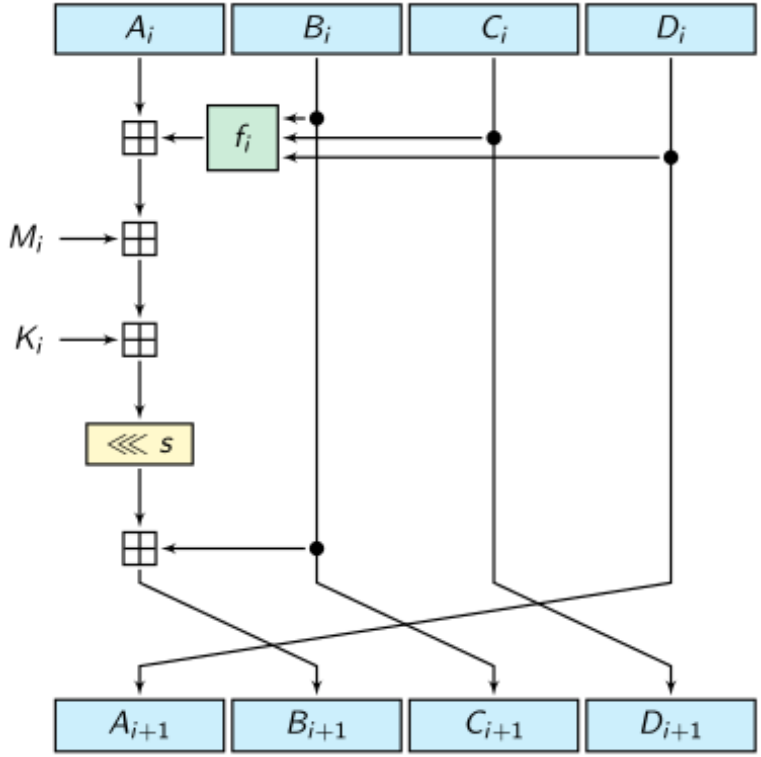
Collision resistance (strong-collision resistant)

Il est difficile de trouver x et x' tels que $x \neq x'$ et

$$h(x) = h(x')$$

MD5 par Ron Rivest en 1991

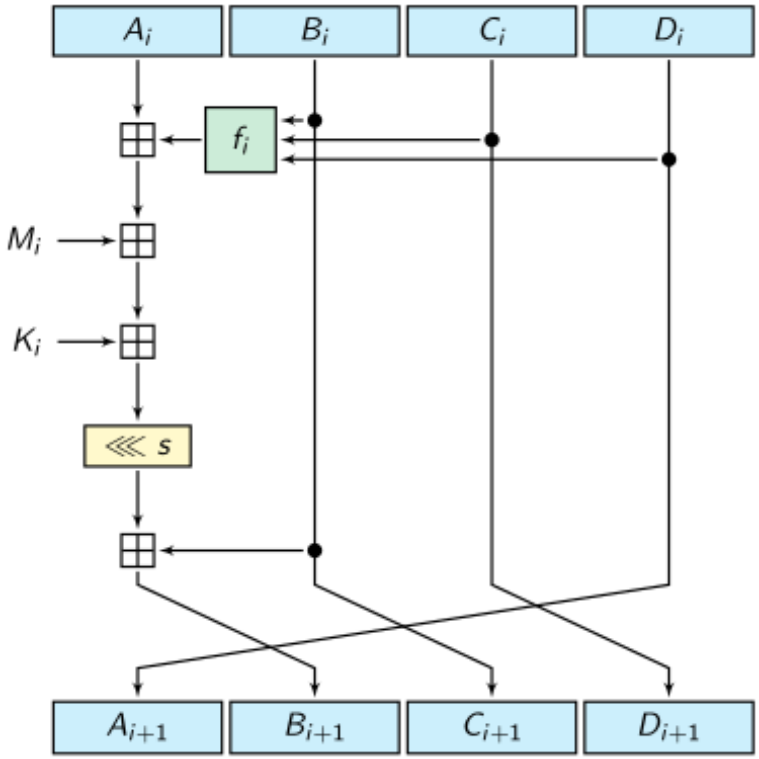
Pour chaque bloc de 512-bit du message à hacher



- K_i constante de 32-bits telle que $K_i^{256} = \lfloor |2^{32} \times \sin(i+1)| \rfloor$
- $\divideontimes$: addition modulo 2^{32}
- <<< s : décalage à gauche de s bits variant à chaque tour

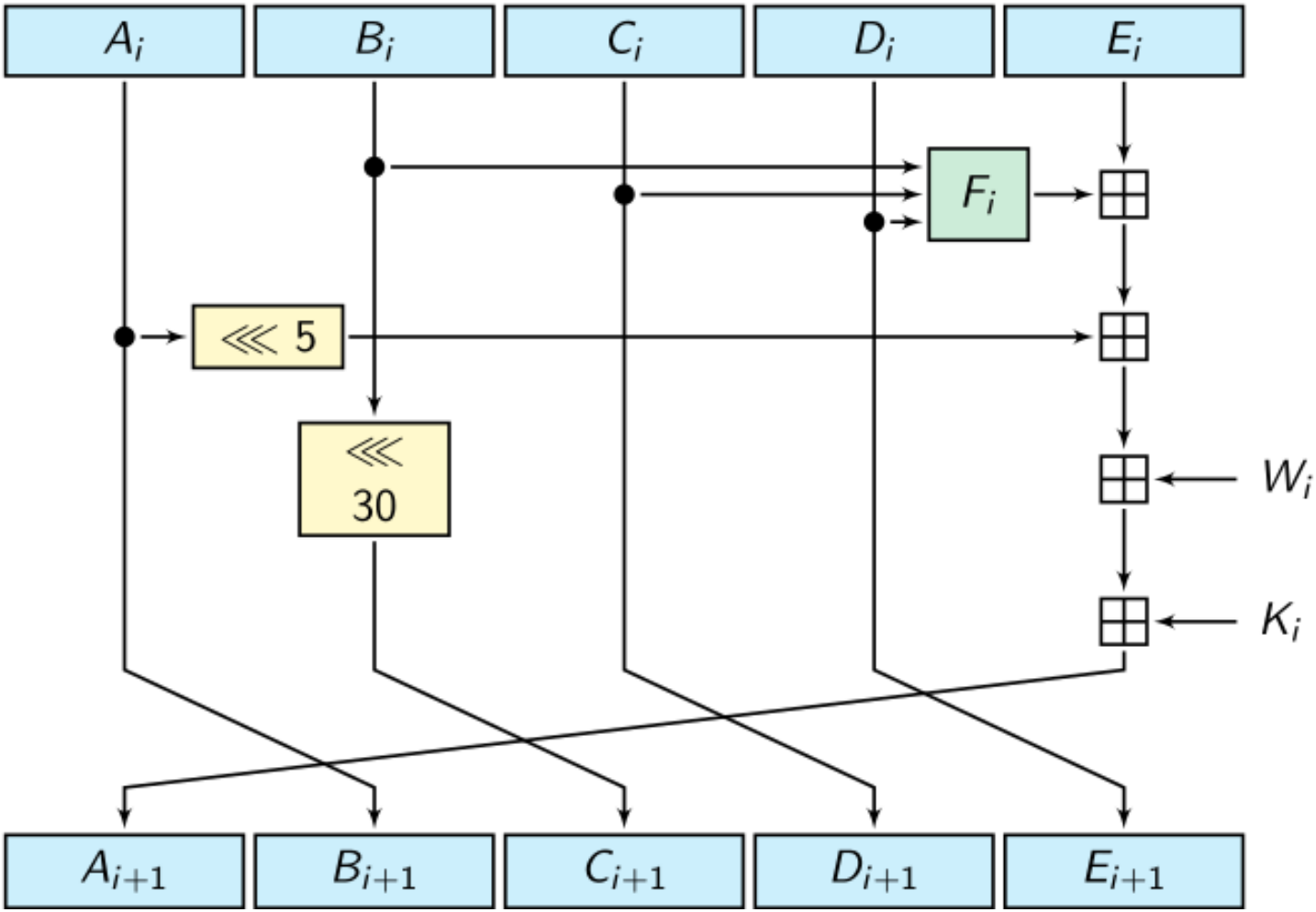
MD5 par Ron Rivest en 1991

MD5 comprend 64 blocs de ce type, groupés en quatre tours de 16 opérations



F est une fonction non linéaire, qui varie selon le tour. M_i symbolise un bloc de 32 bits provenant du message à hacher

SHA-1 (Secure Hash Algorithm), par la NSA en 1995



80 tours

Hash Functions

Algorithm	Output size	Internal state size	Block size	Length size	Word size	Collision
HAVAL	256/.../128	256	1024	64	32	Yes
MD2	128	384	128	No	8	Almost
MD4	128	128	512	64	32	Yes
MD5	128	128	512	64	32	Yes
PANAMA	256	8736	256	No	32	Yes
RadioGatún	Arbitrarily long	58 words	3 words	No	1-64	No
RIPEMD	128	128	512	64	32	Yes
RIPEMD	128/256	128/256	512	64	32	No
RIPEMD	160/320	160/320	512	64	32	No
SHA-0	160	160	512	64	32	Yes
SHA-1	160	160	512	64	32	With flaws
SHA-256/224	256/224	256	512	64	32	No
SHA-512/384	512/384	512	1024	128	64	No
Tiger(2)	192/160/128	192	512	64	64	No
WHIRLPOOL	512	512	512	256	8	No

Historique

$$h : \{0;1\}^* \longrightarrow \{0;1\}^n$$

- MD5 : $n = 128$ (Ron Rivest, 1992)
- SHA-1 : $n = 160$ (NSA, NIST, 1995)
- SHA-2 : $n \in \{224, 256, 384, 512\}$ (NSA, NIST, 2001)
- SHA-3 : n arbitraire (NSA, NIST, 2012)

Si vous avez besoin d'une fonction de hachage, utilisez SHA-3 (éventuellement SHA-2) !

Conclusion

Fonctions de hachage (cryptographique) : primitive extrêmement utile !

- Hachage de mots de passe
- Codes d'authentification de message
- Signature électronique
- Version sécurisée de RSA (RSA-OAEP)
- Blockchain
- etc.

Sommaire

Fonctions de hachage

Sécurisation des mots de passe

TOP 25 en 2014

1. 123456
2. password
3. 12345
4. 12345678
5. qwerty
6. 123456789
7. 1234
8. baseball
9. dragon
10. football
11. 1234567
12. monkey
13. letmein
14. abc123
15. 111111
16. mustang
17. access
18. shadow
19. master
20. michael
21. superman
22. 696969
23. 123123
24. batman
25. trustno1

TOP 25 en 2015

1. 123456 (Unchanged)
2. password (Unchanged)
3. 12345678 (Up 1)
4. qwerty (Up 1)
5. 12345 (Down 2)
6. 123456789 (Unchanged)
7. football (Up 3)
8. 1234 (Down 1)
9. 1234567 (Up 2)
10. baseball (Down 2)
11. **welcome**
12. **1234567890**
13. abc123 (Up 1)
14. 111111 (Up 1)
15. **1qaz2wsx**
16. dragon (Down 7)
17. master (Up 2)
18. monkey (Down 6)
19. letmein (Down 6)
20. **login**
21. **princess**
22. **qwertyuiop**
23. **solo**
24. **passw0rd**
25. **starwars**

TOP 25 en 2021

1. 123456 (Unchanged)
2. 123456789 (Unchanged)
3. qwerty (Up 11)
4. 12345678 (Up 1)
5. 111111 (Up 1)
6. 1234567890 (Up 3)
7. 1234567 (Up 4)
8. password (Down 4)
9. 123123 (Down 3)
10. **987654321**
11. **qwertyuiop**
12. **mynoob**
13. 123321 (Up 10)
14. **666666**
15. **18atcskd2w**
16. **7777777**
17. **1q2w3e4r**
18. 654321 (Up 6)
19. **555555**
20. **3rjs1la7qe**
21. **google**
22. **1q2w3e4r5t**
23. **123qwe**
24. **zxcvbnm**
25. **1q2w3e**

TOP 25 en 2022

1. password (Up 8)
2. 123456 (Down 1)
3. 123456789 (Down 1)
4. **guest**
5. qwerty (Down 2)
6. 12345678 (Down 2)
7. 111111 (Down 2)
8. **12345**
9. **col123456**
10. 123123 (Down 1)
11. 1234567 (Down 4)
12. **1234**
13. **1234567890**
14. **000000**
15. 555555 (Up 4)
16. 666666 (Down 2)
17. 123321 (Down 4)
18. 654321 (Unchanged)
19. 7777777 (Down 5)
20. **123**
21. **D1lakiss**
22. **777777**
23. **110110jp**
24. **1111**
25. 987654321 (Down 15)

Niveau de sécurité : $-\infty$

`identifiant + mot de passe` en **clair** !!!

Interdit !!!

Niveau de sécurité –2

`MD5($pwd)`

```
$query = mysql_query('select * from account where
         login = '.$login.' and pwd = MD5('.$password.')');
```

Peu sécurisé : attaque à l'aide d'une `rainbow table`

- `Rainbow Table` : une table précalculée de valeurs en clair et leurs hash associés

Niveau de sécurité –2

`MD5($pwd)`

Table précalculée (Rainbow Table)		Match avec la BDD leakée
Mot de passe populaire	**MD5**	
123456	e10adc3949ba59abbe56e057f20f883e	mme-michu@yahoo.fr (e10adc3949ba59abbe56e057f20f883e)
azerty	ab4f63f9ac65152575886860dde480a1	raymond@gmail.com (ab4f63f9ac65152575886860dde480a1)
soleil	23206deb7eba65b3fbc80a2ffbc53c28	toto@outlook.com (23206deb7eba65b3fbc80a2ffbc53c28)

mauvaise méthode même avec un `SHA2-256` ou même un `SHA-3`
il faut renforcer la méthode

Salage

Le salage est une méthode permettant de renforcer la sécurité des informations qui sont destinées à être *hachées*

Le salage consiste à concaténer le mot de passe avec une chaîne de caractères avant le hachage

```
$ echo 'message'|sha1sum
1133e3acf0a4cbb9d8b3bfd3f227731b8cd2650b

$ echo 'messagesalé'|sha1sum
641ed04143f3fd810bdd121f6556f6bee0710633
```

Niveau de sécurité −1

```
SHA1($pwd + $salt)
```

- avantage : invalider les `Rainbow Table` existantes
- malheureusement insuffisant

Niveau de sécurité −1

`SHA1($pwd + $salt)`

- avantage : invalider les `Rainbow Table` existantes
- malheureusement insuffisant

Attaque.

- on part d'un utilisateur dont on connaît déjà le mot de passe en clair (un ami ou moi qui ai créé un compte)
- attaque force brute pour retrouver le sel global
- création d'une nouvelle `Rainbow Table`

Niveau de sécurité 0

```
SHA1($pwd + $perUserSalt)
```

- Salage aléatoire par utilisateur
- Une `Rainbow Table` par utilisateur

Niveau de sécurité 1

```
bcrypt($pwd + $perUserSalt)
```

- `bcrypt` : basé sur l'algorithme de chiffrement `Blowfish`
- plus lente à calculer : procédé de brute-force beaucoup plus long
- sel aléatoire : même si l'on retrouve le mot de passe d'un utilisateur, cela ne compromet pas les autres

Niveau de sécurité 2

```
AES(bcrypt($perUserSalt, $pwd), $secretKey)
```

Conclusion

Stockage

- En clair
- Haché (pwd) $\Longrightarrow$ Rainbowtables !
- Haché (pwd + Salt)
- Haché (pwd + Salt-user)
- bcrypt(pwd + Salt-user) bcrypt = hachage plus lent ou PBKDF2
- AES(bcrypt(pwd + Salt-user), SecretKey)